

ROS 2

Flujo de Ejecución: Terminal a Módulo

Jesús Leonardo Cárdenas Martínez
Unidad Tamaulipas
Cinvestav
Ciudad Victoria, México
jesus.cardenas@cinvestav.mx

Abstract—Este documento describe la arquitectura de ejecución de ROS2 (Robot Operating System 2), siguiendo el flujo completo de información desde el comando escrito en la terminal hasta la instanciación de un módulo Python dentro de un nodo.

Se explican los mecanismos internos de cada capa: el entorno de ejecución del sistema operativo, el rol del ejecutable `ros2` como script Python, el sistema de launch files y sus mecanismos de sustitución diferida, el ciclo de vida de arranque de un nodo, el Parameter Server, y el loop de eventos en tiempo real basado en DDS.

El aporte de este documento es que un lector con conocimientos de Python y Linux pero sin experiencia previa en ROS 2 pueda comprender no solo qué ocurre, sino por qué ocurre, y sea capaz de trazar el recorrido de cualquier argumento desde la terminal hasta su uso en el código.

Index Terms—ROS 2, launch files, nodos, variables de entorno, DDS, parámetros

I. EL ENTORNO DE EJECUCIÓN

Antes de que el sistema ROS 2 pueda ser invocado, el sistema operativo debe saber dónde encontrarlo. Esta sección explica la infraestructura sobre la que todo lo demás descansa.

A. Dónde se instala cada distribución

Como se describió en la sección anterior, cada distribución de ROS 2 ocupa su propio directorio dentro de `/opt/ros/`. Si coexistieran Humble y Jazzy instaladas simultáneamente, vivirían en `/opt/ros/humble/` y `/opt/ros/jazzy/` sin interferir entre sí. La distribución activa en una terminal es aquella cuyo `setup.bash` fue sourceado; la variable `ROS_DISTRO` confirma cuál es, como muestra la Terminal 1:

Terminal 1: Verificación de la distribución activa.

```
1 echo $ROS_DISTRO
2 # humble
```

B. Dónde vive ROS 2

ROS 2 Humble se instala en `/opt/ros/humble/`. El directorio `/opt/` es el lugar estándar en Linux para software

opcional instalado en el sistema, es decir, software que no viene con el OS pero que se agrega encima. La Terminal 2 muestra la estructura interna, análoga a la del propio sistema operativo:

Terminal 2: Estructura de instalación de ROS 2 Humble.

```
1 /opt/ros/humble/
2 |-- bin/          <- ejecutables: ros2,
                    rviz2, gazebo...
3 |-- lib/          <- librerías compiladas y
                    paquetes Python
4 |-- include/      <- headers de C++
5 |-- share/        <- recursos y launch files
                    del sistema
6 '-- setup.bash <- el script que se
                    sourcea
```

El workspace del usuario replica exactamente esta estructura pero contiene únicamente los paquetes desarrollados por él, como se ve en la Terminal 3:

Terminal 3: Estructura del workspace tras compilar con `colcon build`.

```
1 ws3/install/
2 |-- control_nodes/
3 |   |-- bin/
4 |   '-- lib/python3.10/site-packages/
5 |-- simulador/
6 '-- setup.bash <- encadenador: sourcea
                    ROS2 + workspace
```

C. Variables de entorno

Una variable de entorno es un par `NOMBRE=valor` almacenado en la memoria del proceso actual. No pertenece a ningún programa en particular, sino al entorno del proceso: la tabla de configuración global que el sistema operativo mantiene para cada proceso en ejecución.

`PATH` es la variable que contiene una lista de directorios separados por `:` donde el OS busca ejecutables. Si `/opt/ros/humble/bin` no está en `PATH`, el comando

ros2 devuelve `command not found`. `PYTHONPATH` cumple la misma función para módulos Python: sin esta variable configurada, la instrucción `import rclpy` falla con `ModuleNotFoundError`. `AMENT_PREFIX_PATH` es utilizada internamente por ROS 2 para localizar los paquetes instalados mediante `ament_index`. Finalmente, `ROS_DISTRO` contiene el nombre de la distribución activa y es leída por algunos paquetes para ajustar su comportamiento. Todas pueden inspeccionarse como indica la Terminal 4:

Terminal 4: Inspección de variables de entorno relevantes.

```
1 echo $PATH          # rutas donde el OS
                      busca ejecutables
2 echo $PYTHONPATH    # rutas donde Python
                      busca módulos
3 echo $ROS_DISTRO    # distribución de ROS2
                      activa
4 env                 # muestra todas las
                      variables
```

D. Qué es `source` y por qué es necesario

Cuando se ejecuta un script de shell de la manera convencional, el OS crea un proceso hijo, ejecuta el script en ese proceso hijo, y al terminar el proceso hijo desaparece llevándose consigo cualquier cambio a las variables de entorno. La terminal del usuario no es afectada. La Terminal 5 ilustra la diferencia:

Terminal 5: Diferencia entre ejecutar y sourcear un script.

```
1 # Ejecutar normal: modifica un proceso
  # hijo que desaparece
2 bash setup.bash      # los cambios a PATH
                      se pierden
3
4 # Sourcear: ejecuta las líneas
  # directamente en la terminal
5 source setup.bash    # los cambios a PATH
                      persisten
```

El comando `source` (equivalente al punto `.` en bash) ejecuta las líneas del script en el proceso actual de la terminal, como si se hubieran escrito directamente. Las modificaciones a `PATH` y `PYTHONPATH` quedan en ese proceso y son heredadas por todos los programas que se lancen desde ahí.

E. Qué hace `setup.bash` internamente

El archivo `ws3/install/setup.bash` generado por `colcon build` actúa como encadenador: primero sourcea ROS 2, luego el workspace del usuario. Su función central es invocar un script Python auxiliar que inspecciona los paquetes instalados en `ws3/install/` y genera dinámicamente los comandos `export` necesarios, como esquematiza la Terminal 6:

Terminal 6: Mecanismo central de `local_setup.bash` (simplificado).

```
1 # El script Python lee ws3/install/ y
  # genera los export
2 _cmds="$(python3 _local_setup_util_sh.py
  sh_bash)"
3
4 # eval ejecuta esas líneas en la terminal
  # actual
5 eval "$$_cmds"
6
7 # El resultado equivale aproximadamente a
  # :
8 export PATH="/ws3/install/control_nodes/
  bin:$PATH"
9 export PYTHONPATH="/ws3/install/
  control_nodes/lib/\
  python3.10/site-packages:$PYTHONPATH"
```

Esto es importante: `source setup.bash` no instala nada ni configura nada de forma permanente. Solo modifica las variables de entorno de la terminal actual. Al cerrar esa terminal, las modificaciones desaparecen. Por eso es necesario sourcear en cada terminal nueva.

F. El shebang y el `PATH`

El ejecutable `ros2` es un archivo de texto. La Terminal 7 muestra su primera línea:

Terminal 7: Primera línea del ejecutable `ros2`.

```
1 #!/usr/bin/env python3
```

Esta línea se llama shebang. Cuando Linux intenta ejecutar un archivo con permisos de ejecución, lee su primera línea: si comienza con `#!`, usa lo que sigue como intérprete y le pasa el archivo como argumento. `/usr/bin/env python3` no apunta a Python directamente; llama a `env`, que busca `python3` en el `PATH` actual. Como ya se sourceó ROS 2, ese `python3` tiene acceso a todos los módulos de ROS 2 a través de `PYTHONPATH`.

En consecuencia, escribir `ros2 launch ...` en la terminal es exactamente equivalente a lo que muestra la Terminal 8:

Terminal 8: Equivalencia entre el comando y su expansión real.

```
1 python3 /opt/ros/humble/bin/ros2 launch \
2     simulador_demo_pva.launch.py \
3     n_rays_used:=5 vmax:=0.8
```

II. EL EJECUTABLE ROS2

A. Cómo el OS entrega los argumentos

Cuando el sistema operativo ejecuta cualquier programa, le entrega todos los tokens escritos en la terminal como una lista de strings llamada `sys.argv`. Para el comando de la Terminal ??, esa lista resultante es la que muestra la Terminal 9:

Terminal 9: `sys.argv` recibido por el ejecutable `ros2`.

```
1 sys.argv = [  
2     'ros2',  
3     'launch',  
4     'simulador',  
5     'demo_pva.launch.py',  
6     'n_rays_used:=5',  
7     'vmax:=0.8'  
8 ]
```

Esta es la frontera más baja del sistema. El OS no sabe si 5 es un entero, un float, o una cadena de texto. Todo es un string. Esta restricción se propaga hacia arriba y obliga a conversiones explícitas de tipo en cada capa.

B. Estructura modular de `ros2`

El ejecutable `ros2` es una CLI (Command Line Interface) modular. Cada subcomando (`launch`, `run`, `topic`, `node`, `param`) es un plugin Python separado que se carga bajo demanda. Al recibir `sys.argv[1] = 'launch'`, el ejecutable carga el plugin `ros2launch` y le delega el resto de la ejecución. Esto permite extender `ros2` con subcomandos personalizados sin modificar el ejecutable principal.

C. Parsing de los argumentos :=

El plugin `ros2launch` utiliza `argparse` de Python para separar el nombre del paquete y el archivo launch. Todo lo que queda después de esos dos campos es tratado como overrides de argumentos del launch file. La convención `clave:=valor` no pertenece a Python ni al OS; es una convención que ROS 2 definió y parsea manualmente mediante string splitting, tal como muestra la Terminal 10:

Terminal 10: Parsing de overrides de argumentos (simplificación del mecanismo interno).

```
1 launch_arguments = {}  
2 for token in remaining_args:  
3     if ':= ' in token:  
4         key, value = token.split(':= ', 1)  
5         launch_arguments[key] = value #  
            siempre string  
6  
7 # Resultado para el comando de la  
   Terminal 1:  
8 # launch_arguments = {'n_rays_used': '5',  
   'vmax': '0.8'}
```

El `split(':= ', 1)` asegura que si el valor contiene `:=` (por ejemplo, una ruta), solo se divide en la primera ocurrencia. El resultado es un diccionario de strings que se entrega al sistema de launch como condiciones iniciales.

D. Localización del launch file con `ament_index`

Una vez parseados los argumentos, `ros2launch` necesita localizar el archivo `demo_pva.launch.py` dentro del paquete `simulador`. Para ello usa `ament_index`: una base de datos de paquetes instalados que vive en `AMENT_PREFIX_PATH`. Cada paquete, al compilarse con `colcon build`, registra su nombre y ubicación en esa base de datos. La Terminal 11 representa lo que `ament_index` hace internamente:

Terminal 11: Localización del launch file (simplificado).

```
1 from ament_index_python.packages import \  
   get_package_share_directory  
2  
3  
4 pkg_dir = get_package_share_directory('simulador')  
5 # -> '/ws3/install/simulador/share/simulador'  
6  
7 launch_file = pkg_dir + '/launch/demo_pva.launch.py'
```

Finalmente, `ros2launch` importa ese archivo como módulo Python normal, mediante `importlib.import_module`, y busca la función `generate_launch_description()`.

III. EL LAUNCH FILE

Un launch file es un script Python cuya única responsabilidad es describir qué nodos lanzar, con qué parámetros y en qué orden. `ros2 launch` lo importa como módulo y busca la función `generate_launch_description()`, que debe retornar un objeto `LaunchDescription`: una lista ordenada de acciones que el sistema procesa en secuencia.

Toda la configuración de parámetros pasa por el mismo par de piezas: `DeclareLaunchArgument` registra el argumento y establece su valor por defecto; `LaunchConfiguration` lo recupera después. El valor *siempre* viaja como string —ROS 2 nunca convierte tipos automáticamente.

Existen dos variantes del patrón según si se necesita o no ejecutar lógica Python sobre los valores.

A. Caso 1: parámetros escalares simples

Cuando los parámetros se pasan directamente al nodo sin transformación, `LaunchConfiguration` puede usarse como sustitución en línea. ROS 2 resuelve su valor en el momento de lanzar el proceso.

Terminal 12: Launch file sin lógica Python. `LaunchConfiguration` actúa como sustitución diferida.

```
1 from launch import LaunchDescription
2 from launch.actions import
  DeclareLaunchArgument
3 from launch.substitutions import
  LaunchConfiguration
4 from launch_ros.actions import Node
5
6 def generate_launch_description():
7     return LaunchDescription([
8         DeclareLaunchArgument('vmax',
9                               default_value='0.5'),
10        DeclareLaunchArgument('wmax',
11                              default_value='1.0'),
12
13        Node(
14            package='control_nodes',
15            executable='consenso_node',
16            namespace='robot0',
17            parameters=[{
18                'vmax':
19                    LaunchConfiguration('vmax'),
20                'wmax':
21                    LaunchConfiguration('wmax'),
22            }],
23        ),
24    ])
```

Este patrón es suficiente cuando no hay cálculo ni conversión de tipos: ROS 2 entrega el string al nodo y el nodo lo interpreta.

B. Caso 2: lógica Python sobre los argumentos

Cuando se necesita procesar el valor en Python —convertir tipos, iterar, calcular retardos— se añade `OpaqueFunction`. Al llegar a esa acción, el sistema llama a la función indicada y le pasa el *context*: el estado global de la ejecución que contiene todos los valores ya resueltos. Dentro de la función, `.perform(context)` extrae el string del context para que el código Python pueda operarlo.

Terminal 13: Launch file con `OpaqueFunction`. Se usa cuando hay conversión de tipos o cálculos sobre los argumentos.

```
1 from launch import LaunchDescription
2 from launch.actions import
  DeclareLaunchArgument, OpaqueFunction
3 from launch.substitutions import
  LaunchConfiguration
4 from launch_ros.actions import Node
5
6 def launch_setup(context, *args, **kwargs):
7     vmax = float(LaunchConfiguration('vmax').perform(context))
8     # Los waypoints llegan como string:
9     # "1.0,0.0,4.0,0.0"
10    raw = LaunchConfiguration('waypoints').perform(context)
11    wps = [float(v) for v in raw.split(',')]
12
13    return [
14        Node(
15            package='control_nodes',
16            executable='navigate_waypoints_pva',
17            namespace='robot0',
18            parameters=[{
19                'vmax': vmax,
20                'waypoints': wps,
21            }],
22        )
23    ]
24
25 def generate_launch_description():
26     return LaunchDescription([
27         DeclareLaunchArgument('vmax',
28                               default_value='0.5'),
29         DeclareLaunchArgument('waypoints',
30                               default_value='4.0,0.0,4.0,4.0,0.0,4.0'),
31         OpaqueFunction(function=launch_setup),
32     ])
```

La diferencia práctica: en el Caso 1, `LaunchConfiguration` es una promesa que ROS 2 resuelve solo; en el Caso 2, `.perform(context)` fuerza la resolución de inmediato para que el código Python pueda operar sobre el valor real.

C. Flujo de un valor de extremo a extremo

El tipo del argumento cambia solo donde el usuario escribe una conversión explícita: `int()` o `float()` en el launch file, e implícitamente al deserializar YAML en el Parameter Server. ROS 2 nunca convierte tipos por cuenta propia en ninguna de las etapas.

IV. ARRANQUE DEL NODO

A. El mecanismo *entry_points*

Cuando el OS ejecuta `navigate_individual_pva`, no ejecuta directamente el archivo Python del usuario. Ejecuta un script wrapper¹ que `setuptools`² generó automáticamente durante `colcon build`, basado en la declaración de la Terminal 14.

Terminal 14: Declaración del ejecutable en `setup.py`.

```
1 entry_points={
2     'console_scripts': [
3         'navigate_individual_pva□=□',
4         'control_nodes.
5             navigate_individual_pva:main'
6     ],
7 }
```

La sintaxis `nombre = modulo:funcion` le dice a `setuptools`: cuando alguien ejecute `navigate_individual_pva`, llama a la función `main()` del módulo `control_nodes.navigate_individual_pva`. La clave `console_scripts`³ es la categoría estándar para esto. Por ello, al agregar un nuevo ejecutable al paquete es necesario recompilar con `colcon build`: el wrapper no existe hasta que se genera.

B. La función *main()*

La Terminal 15 muestra la estructura de la función `main()`:

Terminal 15: Función `main()` típica de un nodo ROS 2.

```
1 def main(args=None):
2     rclpy.init(args=args)
3     node = NavigateIndividual()
4     rclpy.spin(node)
5     node.destroy_node()
6     rclpy.shutdown()
```

`rclpy.init(args=args)` es la primera llamada obligatoria. Inicializa la comunicación con el middleware DDS: establece la conexión con la red de nodos, configura el

¹Un wrapper (envolvente) es un script generado automáticamente que simplemente importa el módulo e invoca la función indicada. Actúa como puente entre el nombre del ejecutable en el sistema de archivos y la función Python real.

²`setuptools` es la biblioteca estándar de Python para empaquetar y distribuir código. ROS 2 la usa para compilar paquetes Python y generar sus ejecutables durante `colcon build`.

³`console_scripts` es la categoría estándar de Python para ejecutables de línea de comandos. Todo lo declarado aquí queda disponible como comando en la terminal después de instalar el paquete.

dominio (`ROS_DOMAIN_ID`), y prepara el Parameter Server⁴ del proceso. Sin esta llamada, ninguna funcionalidad de ROS 2 está disponible: ni suscriptores, ni publicadores, ni parámetros.

`rclpy.spin(node)` entrega el control al loop de eventos⁵ y bloquea el proceso ahí hasta que llegue una señal de terminación (`Ctrl+C` o `SIGINT`⁶). El código del usuario no vuelve a ejecutarse de manera lineal a partir de este punto; solo los callbacks⁷ y timers registrados en `__init__` vuelven a activarse.

C. El constructor *__init__*: secuencia y orden

El constructor del nodo es donde ocurre toda la configuración inicial. No es solo inicialización de variables: es el momento en que el nodo se anuncia en la red ROS 2, declara qué parámetros acepta, y establece todos sus canales de comunicación. El orden de las operaciones importa y no es intercambiable. La Terminal 16 muestra la secuencia completa:

⁴El Parameter Server en ROS 2 no es un proceso externo como en ROS 1. Es una base de datos en memoria interna a cada nodo que almacena sus parámetros con nombre, tipo y valor. Cada nodo tiene el suyo propio.

⁵Un loop de eventos (event loop) es un patrón de programación donde el proceso espera indefinidamente a que ocurran eventos externos (llegada de mensajes, temporizadores) y ejecuta funciones cortas (callbacks) en respuesta. El proceso nunca termina por sí solo; solo sale cuando recibe una señal explícita de terminación.

⁶`SIGINT` (Signal Interrupt) es la señal que el sistema operativo envía a un proceso cuando el usuario presiona `Ctrl+C` en la terminal. ROS 2 captura esta señal internamente y hace que `rclpy.ok()` retorne `False`, lo que provoca que `spin()` termine ordenadamente. Sin este manejo, el proceso se mataría abruptamente sin liberar recursos.

⁷Un callback es una función que no se llama directamente desde el código, sino que se registra para que el sistema la invoque automáticamente cuando ocurre un evento específico. Por ejemplo, `self.odom_callback` se registra para que ROS 2 la llame cada vez que llegue un mensaje de odometría, sin que el programador tenga que verificar manualmente la llegada de datos.

Terminal 16: Constructor completo de un nodo ROS 2 con parámetros, suscriptores y timer.

```
1 class NavigateIndividual(Node):
2     def __init__(self):
3         # 1. Registro en el grafo ROS2
4         super().__init__('navigate_individual')
5
6         # 2. Declaracion de parametros (
7             ANTES de leerlos)
8         self.declare_parameter('
9             n_rays_used', 0)
10        self.declare_parameter('vmax',
11                                1.0)
12
13        # 3. Lectura de parametros ya
14            resueltos
15        n_rays = self.get_parameter('
16            n_rays_used').value
17        vmax = self.get_parameter('vmax
18                                ').value
19
20        # 4. Inicializacion de estado
21            interno
22        self.pose = None
23        self.odom = None
24
25        # 5. Suscriptores
26        self.create_subscription(Odometry
27                                , 'odom',
28                                self.
29                                odom_cb
30                                , 10)
31
32        # 6. Publicadores
33        self.cmd_pub = self.
34            create_publisher(
35                Twist, 'cmd_vel', 10)
36
37        # 7. Modulo de logica (Python
38            puro)
39        self.pva = PVA(n_rays_used=n_rays
40                        , v_max=vmax)
41
42        # 8. Timer de control (al final,
43            cuando todo esta listo)
44        self.create_timer(0.05, self.
45                            control_loop)
```

Por qué este orden. La llamada `super().__init__('navigate_individual')` debe ser la primera: registra el nodo en el grafo ROS 2⁸ con ese nombre y habilita todas las funciones de `self` (`declare_parameter`, `create_subscription`, `get_logger`, etc.). Antes de esa llamada, el objeto existe en Python pero no tiene ninguna capacidad de ROS 2.

⁸El grafo ROS 2 es la representación en tiempo real de todos los nodos activos, sus tópicos, servicios y acciones. Cuando un nodo se registra, los demás nodos pueden descubrirlo automáticamente a través de DDS sin configuración manual. Puede visualizarse con herramientas como `rqt_graph` o `rviz2`.

Los parámetros deben declararse antes de leerse. `get_parameter` solo funciona con parámetros previamente registrados mediante `declare_parameter`; llamarlo antes produce un error.

El timer se crea al final, una vez que todos los suscriptores, publicadores y módulos existen. Si el timer disparara antes de que `self.pva` estuviera construido, el callback intentaría usar un atributo que todavía no existe y Python lanzaría `AttributeError`⁹.

D. Estado interno y el problema del primer ciclo

En programación secuencial el código fluye de arriba hacia abajo: una instrucción se ejecuta después de la anterior. En un nodo ROS 2 no hay un flujo lineal después de `spin`: el programa se vuelve orientado a eventos¹⁰. El estado del nodo vive en variables de instancia (`self.pose`, `self.odom`) que los callbacks de suscripción actualizan cuando llegan mensajes, y que el timer lee cuando dispara.

El problema es que el timer puede disparar antes de que haya llegado el primer mensaje. Por eso `self.pose = None`¹¹: el valor `None` actúa como centinela que señala que aún no hay información disponible. El callback de control debe verificarlo antes de operar, como muestra la Terminal 17:

Terminal 17: Verificación de datos disponibles al inicio del loop de control.

```
1 def control_loop(self):
2     if self.pose is None or self.odom is
3         None:
4         return # todavia no llego
5             informacion, esperar
6
7     # a partir de aqui, self.pose y self.
8         odom son validos
9     vel = self.pva.compute(self.pose,
10                            self.odom)
11     self.cmd_pub.publish(vel)
```

Sin esta verificación, el nodo fallaría en los primeros ciclos de control con un `AttributeError` o `TypeError`.

⁹`AttributeError` es la excepción de Python que se lanza al intentar acceder a un atributo de un objeto que no existe. Por ejemplo, `self.pva.compute()` fallaría con `AttributeError` si `self.pva` aún no fue asignado.

¹⁰La programación orientada a eventos es un paradigma donde el flujo del programa está determinado por eventos externos (mensajes, temporizadores) en lugar de por una secuencia predefinida de instrucciones. En lugar de preguntar “¿llegó un mensaje?” cada ciertos milisegundos (sondeo o polling), el nodo registra callbacks y el sistema lo despierta solo cuando ocurre algo relevante (notificación o event-driven). Esto ahorra CPU porque el proceso duerme mientras no hay eventos.

¹¹`None` es el valor especial de Python que representa ausencia de dato. Aquí se usa como valor centinela (sentinel): un marcador artificial que indica que una variable aún no ha recibido un valor real. Si todavía vale `None`, es porque el suscriptor no ha recibido ningún mensaje aún.

Es una fuente frecuente de bugs en nodos ROS 2 nuevos que parece funcionar en pruebas (porque los mensajes ya llegaron) y falla al arrancar en frío.

E. Declaración y lectura de parámetros en detalle

La Terminal 18 ilustra el mecanismo completo:

Terminal 18: Declaración y lectura de parámetros en el nodo.

```
1 self.declare_parameter('n_rays_used', 0)
2 self.declare_parameter('vmax', 1.0)
3
4 n_rays = self.get_parameter('n_rays_used')
   .value # int 5
5 vmax = self.get_parameter('vmax').value
   # float 0.8
```

El segundo argumento de `declare_parameter` es el valor por defecto y también define el tipo esperado. Si se declara ('vmax', 1.0) (float) y desde la terminal se pasa `vmax:=texto`, ROS 2 lanzará un error de tipo. El tipo del default funciona como contrato¹²: el Parameter Server solo acepta valores del mismo tipo.

`get_parameter('vmax').value` accede al Parameter Server y retorna el valor en el tipo Python correspondiente: `int`, `float`, `str`, `bool` o `list`. Si desde la terminal llegó `vmax:=0.8`, el YAML lo deserializó a `float(0.8)`, y `.value` lo entrega ya convertido.

F. Las tres fronteras de serialización

El valor 5 escrito en la terminal recorre tres conversiones antes de estar disponible en Python. Cada conversión es un punto donde el tipo del dato puede cambiar o perderse, lo que explica muchos errores comunes en ROS 2:

TABLE I: Fronteras de serialización del argumento `n_rays_used:=5`.

Frontera	De	A	Mecanismo
Terminal → launch	string '5'	string '5'	<code>split(':')</code>
Launch → proceso	Python int 5	YAML 5	<code>-ros-args -p</code>
Proceso → nodo	YAML 5	Python int 5	<code>.value</code>

En la primera frontera no hay conversión: todo es string. La conversión a `int` ocurre en el launch file, donde el usuario la escribe explícitamente con `int(...)`. En la tercera frontera, el YAML se deserializa automáticamente y el tipo se infiere del valor declarado como default. Esto significa que el tipo final del parámetro depende de dos decisiones separadas: el `int()` en el launch file y el default en `declare_parameter()`. Si no coinciden, pueden ocurrir errores difíciles de depurar.

¹²Un contrato de tipo significa que una vez declarado un parámetro como float, el Parameter Server rechaza cualquier intento de asignarle un valor de otro tipo. Esto evita errores silenciosos donde un parámetro cambia de tipo inesperadamente.

V. DEL NODO AL MÓDULO

A. Instanciación de módulos Python puros

Una vez recuperados los valores del Parameter Server, el nodo los usa para construir los objetos que implementan la lógica de control. En este punto el código abandona por completo el dominio de ROS 2. La Terminal 19 muestra cómo se instancia el módulo PVA:

Terminal 19: Instanciación del módulo PVA desde el nodo.

```
1 from control_nodes.algorithms.pva import
   PVA
2
3 self.pva = PVA(
4     n_rays_used=n_rays, # int 5
5     v_max=vmax, # float 0.8
6     d_safe=self.get_parameter('d_safe').
   value,
7     d_influence=self.get_parameter('
   d_influence').value,
8     xi=self.get_parameter('xi').value,
9     rp=0.25,
10 )
```

`n_rays` es un entero Python normal. La clase PVA no sabe nada de ROS 2, no sabe que existe un launch file, no sabe que hubo una terminal. Recibe un argumento de constructor como cualquier clase Python.

B. La clase PVA como Python puro

La Terminal 20 muestra el constructor de la clase PVA, que es Python estándar sin ninguna dependencia de ROS 2:

Terminal 20: Constructor de la clase PVA.

```
1 class PVA:
2     def __init__(self,
3                 d_safe=0.3,
4                 d_influence=1.0,
5                 xi=1.0,
6                 rp=0.25,
7                 v_max=0.5,
8                 w_max=0.5,
9                 n_rays_used=None):
10
11         self.d_safe = d_safe
12         self.d_influence = d_influence
13         self.xi = xi
14         self.rp = rp
15         self.v_max = v_max
16         self.w_max = w_max
17         self.n_rays_used = n_rays_used
```

Desde aquí en adelante es orientación a objetos Python estándar. La separación entre el código de ROS 2 (el nodo) y la lógica de la aplicación (el módulo) es una práctica

arquitectónica deliberada: permite probar la clase PVA de forma independiente, sin necesitar un contexto ROS 2 activo.

C. Por qué esta separación es importante

El nodo actúa como adaptador: su responsabilidad es recibir datos de la infraestructura ROS 2 (tópicos, parámetros) y entregarlos a los módulos de lógica de control en los tipos correctos. Los módulos implementan los algoritmos sin acoplarse al middleware. Esto tiene consecuencias concretas. En cuanto a testabilidad, los módulos pueden probarse con `unittest` sin inicializar `rclpy`. En cuanto a portabilidad, el mismo módulo puede usarse en otro sistema de middleware cambiando solo el nodo adaptador. En cuanto a claridad, la complejidad de ROS 2 queda confinada al nodo y la lógica de control es matemáticamente pura.

VI. EJECUCIÓN EN TIEMPO REAL

A. El loop de eventos: `rclpy.spin()`

Una vez que el nodo está construido y todos sus suscriptores, publicadores y timers registrados, la llamada `rclpy.spin(node)` entrega el control al loop de eventos de ROS 2. La Terminal 21 muestra su comportamiento conceptual:

Terminal 21: Comportamiento conceptual de `rclpy.spin()`.

```
1 # Pseudocódigo del loop de eventos
2 while rclpy.ok():
3     esperar_evento()           # bloquea
4     ejecutar_callback()       # procesa
5     # repetir
```

El proceso permanece bloqueado en este loop indefinidamente. El código del usuario solo vuelve a ejecutarse cuando algún evento lo activa: la llegada de un mensaje en un tópico suscrito, o el disparo de un timer. Fuera de esos momentos, el proceso duerme y no consume CPU.

B. Suscripciones y callbacks de mensajes

Una suscripción registra un callback que ROS 2 llama automáticamente cada vez que llega un mensaje en el tópico indicado, como muestra la Terminal 22:

Terminal 22: Registro de una suscripción a odometría.

```
1 self.odom_sub = self.create_subscription(
2     Odometry,           # tipo del
3     'odom',             # nombre del
4     self.odom_callback, # funcion a
5     10                  # queue size (QoS
6     History)
```

El argumento 10 es el tamaño de la cola: si los mensajes llegan más rápido de lo que el callback los procesa, se acumulan hasta 10 mensajes antes de empezar a descartarlos. DDS gestiona la entrega desde el publicador (el nodo de odometría de Gazebo) hasta este callback, incluso si están en procesos diferentes.

El callback de suscripción típicamente solo actualiza el estado interno del nodo:

Terminal 23: Callback de suscripción: actualiza estado, no computa control.

```
1 def odom_callback(self, msg: Odometry):
2     self.odom = msg # guarda el mensaje
3     # NO computar control aqui: llega a
4     # frecuencia variable
```

La razón de no computar el control dentro del callback de suscripción es que los mensajes de odometría llegan a la frecuencia que el publicador decida (o que la red permita), no a la frecuencia que el algoritmo de control necesita. Mezclar la lógica de control con la llegada de datos produce un nodo con frecuencia de operación impredecible.

C. Timers: el ciclo de control

Un timer registra un callback que se ejecuta periódicamente con frecuencia determinista. La Terminal 24 muestra cómo crear el loop de control a 20 Hz:

Terminal 24: Registro de un timer de control a 20 Hz.

```
1 self.timer = self.create_timer(
2     0.05,           # periodo en
3     self.control_loop # funcion a
4     llamar
```

El timer garantiza que `control_loop` se ejecute cada 50 ms independientemente de cuándo llegaron los últimos

mensajes. Esta separación entre la *adquisición de datos* (callbacks de suscripción) y el *cómputo de control* (callback del timer) es el patrón fundamental en nodos de control con ROS 2.

D. El patrón completo: adquisición y control separados

La Terminal 25 muestra el patrón completo de un nodo de control tal como se estructura en este proyecto:

Terminal 25: Patrón completo de nodo de control: suscriptores actualizan estado, timer computa y publica.

```

1 def odom_callback(self, msg):
2     self.odom = msg          # actualiza
                               estado: 50-100 Hz
3
4 def scan_callback(self, msg):
5     self.scan = msg          # actualiza
                               estado: 10 Hz
6
7 def control_loop(self):      # ejecuta a
                               20 Hz exactos
8     # guardia: datos aun no disponibles
9     if self.odom is None or self.scan is
       None:
10        return
11
12    # computo de control con datos mas
       recientes
13    cmd = self.pva.compute(self.odom,
                             self.scan)
14
15    # publicacion del comando de
       velocidad
16    self.cmd_pub.publish(cmd)

```

Este patrón resuelve el problema de la asincronía inherente a los sistemas distribuidos: el nodo no bloquea esperando datos, sino que usa los datos más recientes disponibles en el momento en que el timer dispara. Si el LiDAR llega a 10 Hz y el control corre a 20 Hz, cada par de ciclos de control usará el mismo scan, lo cual es completamente aceptable.

E. Modelo de concurrencia: SingleThreadedExecutor

Por defecto, `rclpy.spin()` utiliza un `SingleThreadedExecutor`, lo que significa que todos los callbacks (suscriptores y timers) se ejecutan en el mismo hilo, uno a la vez. Esto tiene dos consecuencias:

Primero, el acceso a variables compartidas como `self.odom` y `self.scan` es seguro sin necesidad de locks: el executor garantiza que nunca habrá dos callbacks ejecutándose simultáneamente. Segundo, si un callback tarda más que su periodo (por ejemplo, el loop de control tarda 60 ms siendo el periodo 50 ms), el siguiente disparo del timer se retrasa. Los callbacks deben ser lo suficientemente rápidos para no acumular retraso.

Si se necesita paralelismo real (procesamiento de imagen pesado mientras el control sigue corriendo), existe `MultiThreadedExecutor` con `ReentrantCallbackGroup`, pero requiere manejo explícito de acceso concurrente a las variables compartidas.

F. El rol de DDS en la entrega de mensajes

DDS es la capa de transporte subyacente de ROS 2. Cuando un nodo arranca, DDS anuncia su existencia en la red y otros nodos que suscriben al mismo tópico se conectan automáticamente, sin configuración manual de IPs o puertos. Para la transmisión, DDS serializa los mensajes al formato wire, los transmite (localmente vía memoria compartida, o en red vía UDP) y los deserializa en el receptor.

El usuario de ROS 2 no interactúa con DDS directamente; `rclpy` lo abstrae completamente. Sin embargo, entender que DDS existe explica por qué los nodos se descubren automáticamente y por qué es posible comunicar nodos en máquinas diferentes con solo configurar `ROS_DOMAIN_ID`.

VII. RESUMEN DEL FLUJO COMPLETO

La Tabla III condensa el recorrido completo del argumento `n_rays_used:=5` a través de las cuatro capas del sistema.

TABLE II: Flujo completo del argumento `n_rays_used:=5` desde la terminal hasta el módulo.

Capa	Forma del valor	Mecanismo	Responsable
Terminal	string '5'	<code>sys.argv</code> <code>split(':=')</code>	OS / ros2
Launch file	string '5' → int 5	<code>.perform(ctx)</code> <code>+ int(...)</code>	Usuario
Proceso	YAML <code>n_rays:</code> 5	<code>-ros-args -p</code>	ROS 2
Nodo	int 5	<code>get_parameter</code> <code>.value</code>	rclpy
Módulo	int 5	argumento constructor	Python

El diagrama siguiente ilustra las entidades involucradas en cada nivel y los mecanismos de cruce entre ellos:

Terminal 26: Diagrama de flujo Terminal → Módulo.

```
1  TERMINAL
2  "n_rays_used:=5_vmax:=0.8"
3  | sys.argv + split(':=')
4  v
5  LAUNCH FILE [generate_launch_description
6  ]
7  DeclareLaunchArgument  <- registra y
   almacena
8  OpaqueFunction         <- logica
   Python
9  .perform(context)      <- resuelve el
   string
10 int('5') = 5           <- conversion:
   usuario
11 Node(parameters={...}) <- descriptor (
   no proceso)
   | serializa a YAML, pasa como --
   ros-args -p
12 v
13 PROCESO [navigate_individual_pva --ros-
   args ...]
14 rclpy.init()           <- inicia DDS
15 Node.__init__()        <- registra en
   el grafo
16 declare_parameter()    <- almacena en
   param server
17 get_parameter().value  <- recupera
   como int
18 | argumento de constructor Python
   normal
19 v
20 MODULO [class PVA]
21 self.n_rays_used = 5    <- Python puro
```

A. Observaciones finales

Tres conclusiones emergen del análisis de este flujo.

Todo empieza como string. La restricción del OS de que `sys.argv` es siempre una lista de strings se propaga hacia arriba. ROS 2 nunca convierte tipos automáticamente. Esa responsabilidad recae en el usuario dentro del launch file.

Cada capa tiene su propio mecanismo de serialización. El cruce entre capas requiere una conversión de formato: string splitting en la terminal, YAML al pasar al proceso, deserialización en el Parameter Server. Comprender estos mecanismos permite depurar errores de tipo que de otro modo son opacos.

El módulo de lógica no sabe que existe ROS 2. La arquitectura de separación nodo-módulo no es accidental: confina la complejidad del middleware a una capa de adaptación y deja el código de control matemáticamente puro, testeable y portable.